Davante

16

PROGRAMACIÓN

Colecciones

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Introducción a las colecciones 5

/ 3. Interfaz Collection 5

/ 4. Caso práctico 1: "¿Usar un array o ArrayList?" 7

/ 5. Tipos genéricos 7

5.1. Interfaces genéricas 8

5.2. Métodos genéricos 9

/ 6. Listas: Interfaz List 10

6.1. ArrayList 11

6.2. LinkedList 12

/ 7. Conjuntos: Interfaz Set 13

7.1. HashSet 14

7.2. TreeSet 14

7.3. LinkedHashSet 15

/ 8. Interfaz Map 16

/ 9. Caso práctico 2: "Eliminar elementos de la colección" 17

/ 10. Colas: Interfaz Queue 18

ÍNDICE

/ 11. Iteradores 19

/ 12. Resumen y resolución del caso práctico de la unidad 20

/ 13. Bibliografía 21

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Diferenciar los tipos de colecciones.

Utilizar clases genéricas.

Reconocer cuándo utilizar los diferentes tipos de colecciones.

Saber emplear iteradores.

/ 1. Introducción y contextualización práctica

En este tema hablaremos sobre las colecciones. Es un concepto bastante importante, ya que permitirá realizar programas más complejos de una forma mucho más sencilla.

En Java existen multitud de colecciones y no sería viable estudiarlas todas en esta unidad, no obstante, vamos a aprender y profundizar sobre las más utilizadas.

También se definirá el concepto de clases genéricas y la gran utilidad que estas pueden llegar a generar.

Por último, se verá cómo recorrer cualquier tipo de colección de una forma segura mediante el uso de iteradores.

A continuación, se presenta un caso práctico a través del cual podremos aproximarnos de forma aplicada a la teoría de este tema.



Fig.1. Colección de figuras.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "¿Qué colección utilizo?"

https://on.soundcloud.com/QrD9KWKSbFdjbXaF6

/ 2. Introducción a las colecciones

Cuando hablamos de **colecciones** en programación, hacemos referencia a un **conjunto de elementos que se encuentran almacenados de forma conjunta en una misma estructura de datos**.

Estos elementos van a potenciar la reusabilidad de código, ya que, al poder crear colecciones con una misma estructura, pero con diferentes tipos, podremos aplicar lo que se conoce como **métodos genéricos**, o, lo que es lo mismo, métodos que van a pertenecer a la colección y que se podrán ejecutar de forma independiente del tipo de dato que contenga dicha colección.

La única restricción que tienen las colecciones es que los **objetos** que almacenan en su interior han de ser **del mismo tipo o de tipos relacionados**.

Se podría decir, por tanto, que las colecciones son estructuras de datos dinámicas que permiten almacenar y manipular grupos de objetos del mismo tipo proporcionando diferentes formas de almacenar elementos, como listas, conjuntos y mapas.

Las colecciones están disponibles en muchos otros lenguajes de programación, como C++ o Python. En Java, para poder utilizarlas, necesitamos la clase Collection y el siguiente import:

```
importjava.util.Collection;
```

Los métodos genéricos que se emplearán son los que se muestran en la siguiente tabla:

Método	Descripción
intsize()	Devuelve la cantidad de elementos que tiene la colección.
Void add(Object ob)	Agrega a la colección el elemento que se pasa por parámetro.
Void addAll(Collection c)	Agrega a la colección todos los elementos de la colección que se pasa por parámetro.
Boolean remove(Object ob)	Elimina de la colección el elemento que se pasa por parámetro.
Boolean is Empty()	Elimina todos los elementos de la colección que se encuentran en la colección que se pasa por parámetro.
voidclear()	Elimina todos los elementos de la colección.

Tabla 1. Métodos comunes de las colecciones.

/ 3. Interfaz Collection

Ya se ha visto que todas las colecciones hacen uso de un conjunto de métodos genéricos. Para su implementación, Java proporciona un conjunto de estructuras que comparten dicho conjunto de métodos, es decir, una interfaz. Dicha interfaz se denomina Collection.

La interfaz Collection representa una **colección de objetos cuyo propósito principal es proporcionar métodos comunes para manipular y trabajar con colecciones de elementos en Java**.

La siguiente figura muestra los tipos fundamentales de estructuras que implementa la interfaz:

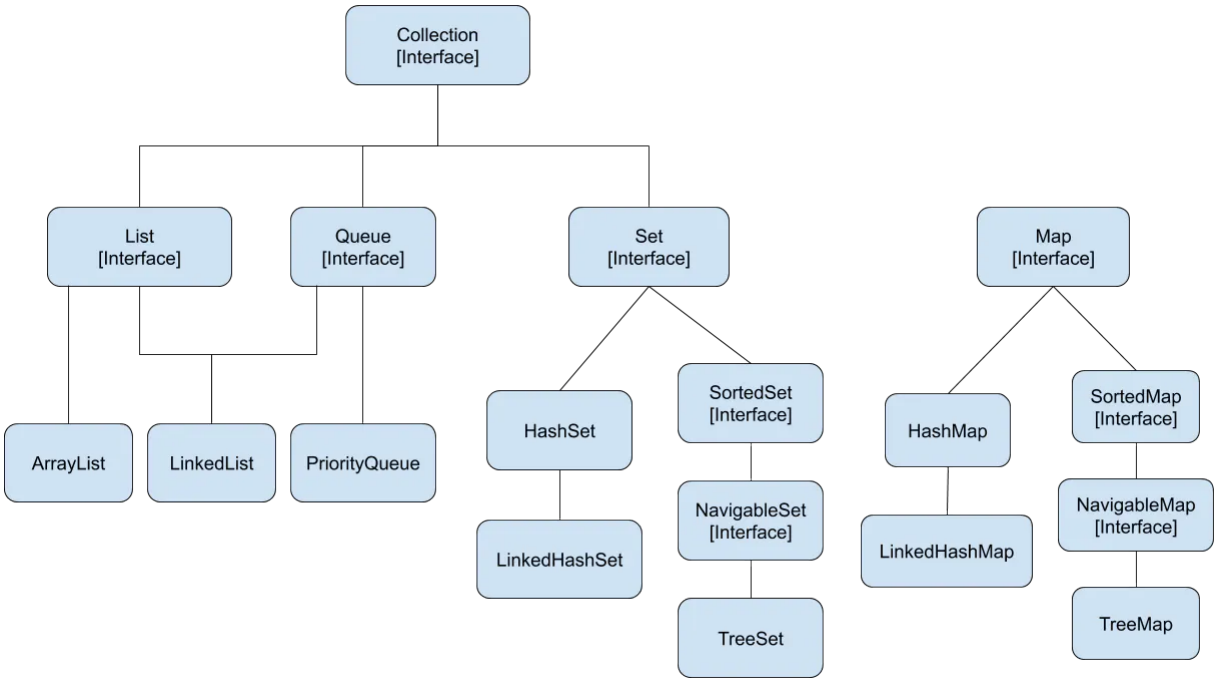


Fig.2. Interfaz Collection.

Como se puede observar en la imagen, la interfaz Collection es una **interfaz raíz** en la jerarquía de colecciones de Java. Otros tipos de colecciones, como List, Set y Queue, extienden esta interfaz:

- **Lista (List)**: Es una colección ordenada que permite elementos duplicados. Sus elementos están indexados y se puede acceder a ellos por su posición. Algunas implementaciones comunes de la interfaz List son ArrayList, LinkedList y Vector.

- **Conjunto (Set)**: Es una colección que no permite elementos duplicados. No tienen un orden específico de elementos. Algunas implementaciones comunes de la interfaz Set son HashSet, TreeSet y LinkedHashSet.

- **Cola (Queue)**: Es una colección que sigue el principio FIFO (First In, FirstOut). Los elementos se insertan al final y se eliminan del principio. Algunas implementaciones comunes de la interfaz Queue son LinkedList, PriorityQueue, etc.

- **Mapa (Map)**: Es una colección de pares clave-valor, donde cada clave está asociada con un único valor. Las claves no pueden ser duplicadas, pero los valores pueden serlo. Algunas implementaciones comunes de la interfaz Map son HashMap, TreeMap, LinkedHashMap, etc.

El uso de las colecciones no garantiza un orden específico para los elementos. Estas pueden contener elementos duplicados, por lo que una de sus principales características es la definición de métodos para agregar elementos, eliminar elementos, verificar la existencia de elementos, obtener el tamaño de la colección, entre otros.

/ 4. Caso práctico 1: "¿Usar un array o ArrayList?"

Planteamiento: Tanto los *arrays* como los ArrayList son estructuras de datos importantes, pero se utilizan en diferentes situaciones. Los arrays son adecuados cuando se conoce de antemano el tamaño fijo de los elementos y se necesita un acceso rápido a los mismos. Son más eficientes en términos de memoria y rendimiento, ya que los elementos se almacenan de forma contigua en la memoria. Sin embargo, no pueden cambiar de tamaño una vez que se han creado, lo que puede ser limitante en algunas situaciones. Por otro lado, los ArrayList son una implementación de la interfaz List en Java y ofrecen una mayor flexibilidad en términos de tamaño dinámico. Permiten agregar, eliminar y modificar elementos de manera más sencilla, ya que se encargan automáticamente de redimensionarse según sea necesario.

Nudo: ¿Es mejor utilizar arrays convencionales o utilizar la clase ArrayList?

Desenlace:

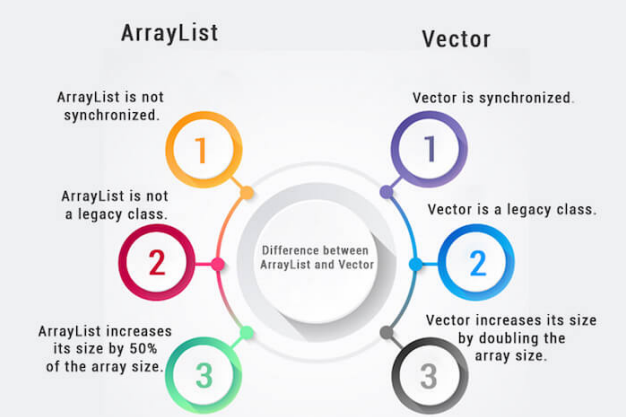


Fig.3. Diferencias entre array y ArrayList.

La clase ArrayList, como clase que es, ya tiene implementadas una serie de funciones y una estructura interna totalmente probada y que funciona correctamente. Esto va a hacer que un problema complicado de resolver mediante arrays convencionales, si conocemos las funciones de la clase ArrayList, sea mucho más fácil de implementar.

Entre las acciones que podríamos hacer de una forma mucho más cómoda con ArrayList, podrían estar ordenar el array, recorrerlo, buscar un elemento dentro de él, etc.

En resumen, se recomienda utilizar arrays cuando se conoce el tamaño fijo de los elementos y se necesita un acceso rápido, mientras que los ArrayList son más adecuados cuando se requiere un tamaño dinámico y una manipulación más flexible de los elementos.

/ 5. Tipos genéricos

Ya se ha dicho que las colecciones emplean métodos genéricos, pero ¿a qué se hace referencia exactamente? En Java existe lo que se llaman tipos genéricos o tipos parametrizados. Este tipo de datos tienen gran relevancia y pueden llegar a ser muy útiles, ya que van a permitir **crear funcionalidades donde se pueda elegir el tipo de dato** con el que se va a operar.

Concretamente, van a permitir crear **plantillas de funcionalidades** donde **el tipo de dato con el que operen vendrá dado por parámetro**, lo que constituirá su mayor ventaja. Trabajan directamente con el tipo que se le pase por parámetro, sin necesidad de hacer *castings* ni que los tipos sean compatibles entre sí.

En el uso de los genéricos, se podrán emplear solamente objetos, es decir, los **tipos primitivos** (como int, char, double, etc.), no se pueden utilizar directamente como argumentos para parámetros de tipo en clases genéricas. Esto se debe a que Java solo admite tipos de referencia como argumentos de tipo en clases genéricas.

Sin embargo, para manejar tipos primitivos en clases genéricas, Java proporciona **clases envolventes** (*wrapper classes*) que encapsulan los tipos primitivos. Estas clases envolventes son:

- Integer para int.
- Long para long.
- Short para short.
- Byte para byte.
- Float para float.
- Double para double.
- Character para char.
- Boolean para boolean.

La figura 4 muestra una clase de tipo genérico:

```
public class MiClase<T> {
    private T miObjeto;

    public MiClase(T miObjeto) {
        this.miObjeto = miObjeto;
    }

    public T getMiObjeto() {
        return miObjeto;
    }

    public void setMiObjeto(T miObjeto) {
        this.miObjeto = miObjeto;
    }

    public void mostrarTipo() {
        System.out.println("El tipo de T es: " + miObjeto.getClass().getName());
    }

    public static void main(String[] args) {
        // Crear una instancia de MiClase con un tipo específico
        MiClase<Integer> miEntero = new MiClase<>(10);
        miEntero.mostrarTipo(); // Output: El tipo de T es: java.lang.Integer
        System.out.println("Mi objeto es: " + miEntero.getMiObjeto()); // Output: 10

        // Crear una instancia de MiClase con otro tipo
        MiClase<String> miString = new MiClase<>("Hola mundo");
        miString.mostrarTipo(); // Output: El tipo de T es: java.lang.String
        System.out.println("Mi objeto es: " + miString.getMiObjeto()); // Output: Hola mundo
    }
}
```

Fig.4. Clase genérica Integer.

En esta clase, MiClase, T es un parámetro de tipo genérico. Podemos usar este parámetro para definir el tipo de datos de los campos, métodos y constructores de la clase. Al crear una instancia de MiClase, proporcionamos el tipo concreto que deseamos utilizar entre los corchetes angulares (<>), también llamados operador diamante.

5.1. Interfaces genéricas

Las interfaces genéricas en Java **permiten definir interfaces que pueden trabajar con tipos de datos genéricos**. Esto proporciona flexibilidad, al diseñar componentes que pueden trabajar con diferentes tipos de datos de manera segura y sin necesidad de realizar conversiones de tipo.

El siguiente código (figura 5) muestra un ejemplo de interfaz genérica:

```java
// Definición de una interfaz genérica
interface MiInterfaz<T> {
    void mostrarTipo(T obj);
}

// Implementación de la interfaz genérica con un tipo concreto
class ImplementacionInterfaz implements MiInterfaz<Integer> {
    // Implementación del método de la interfaz
    public void mostrarTipo(Integer obj) {
        System.out.println("El tipo es: " + obj.getClass().getName());
    }
}

public class EjemploInterfacesGenericas {
    public static void main(String[] args) {
        // Crear una instancia de la implementación de la interfaz
        ImplementacionInterfaz obj = new ImplementacionInterfaz();

        // Llamar al método de la interfaz con un tipo concreto
        obj.mostrarTipo(10); // Output: El tipo es: java.lang.Integer
    }
}
```

Fig.5. Interfaz genérica.

En este ejemplo, MiInterfaz es una interfaz genérica que tiene un parámetro de tipo T. La implementación ImplementacionInterfaz especifica que va a trabajar con objetos de tipo Integer. Cuando se llama al método mostrarTipo de la interfaz con un objeto Integer, imprime el nombre de la clase del objeto.

5.2. Métodos genéricos

Los métodos genéricos en Java **permiten escribir métodos que pueden trabajar con diferentes tipos de datos de manera segura**, sin tener que especificar el tipo de datos en el momento de escribir el método. Esto proporciona flexibilidad y reutilización de código, al escribir algoritmos que funcionan con múltiples tipos de datos.

```java
public class MetodosGenericos {

    // Método genérico para intercambiar dos elementos en un array
    public static <T> void intercambiar(T[] array, int i, int j) {
        if (i >= 0 && i < array.length && j >= 0 && j < array.length) {
            T temp = array[i];
            array[i] = array[j];
            array[j] = temp;
        } else {
            System.out.println("Índices fuera de rango.");
        }
    }

    public static void main(String[] args) {
        // Ejemplo de uso del método genérico
        Integer[] arrayEnteros = {1, 2, 3, 4, 5};
        System.out.println("Array original: " + java.util.Arrays.toString(arrayEnteros));

        // Intercambiar elementos en el array
        intercambiar(arrayEnteros, 1, 3);
        System.out.println("Array después del intercambio: " + java.util.Arrays.toString(arrayEnteros));

        // Ejemplo con un array de Strings
        String[] arrayStrings = {"Hola", "Mundo", "Java"};
        System.out.println("Array original: " + java.util.Arrays.toString(arrayStrings));

        // Intercambiar elementos en el array de Strings
        intercambiar(arrayStrings, 0, 2);
        System.out.println("Array después del intercambio: " + java.util.Arrays.toString(arrayStrings));
    }
}
```

Fig.6. Métodos genéricos.

En el ejemplo de código anterior (figura 6), el método intercambiar es un método genérico que puede trabajar con cualquier tipo de array (T[]). Dentro del método, se utiliza el tipo genérico T para intercambiar los elementos en las posiciones i y j del array.

Al llamar al método intercambiar, se puede pasar cualquier tipo de array (por ejemplo, un array de enteros Integer[] o un array de cadenas String[]) y el método funcionará correctamente. Java deduce automáticamente el tipo de datos en función del contexto en el que se utiliza el método genérico.



Vídeo 1. "Creación y uso de una clase genérica"

https://bit.ly/2DsFLc2



/ 6. Listas: Interfaz List

La interfaz List en Java representa una **colección ordenada e indexada que permite elementos duplicados**.

Algunos de los **métodos comunes** proporcionados por la interfaz List incluyen:

- **add(E elemento)**: Se utiliza para agregar un elemento al final de la lista tomando como argumento el elemento que se va a agregar y devolviendo 'true' si el elemento se agregó correctamente.
- **remove(Object objeto)**: Se emplea para eliminar la primera ocurrencia del objeto especificado de la lista tomando como argumento el objeto que se va a eliminar y devolviendo 'true' si se eliminó correctamente.
- **get(intindice)**: Se usa para obtener el elemento en la posición especificada por el índice tomando como argumento el índice del elemento que se desea obtener y devolviendo el elemento en la posición especificada.
- **set(intindice, E elemento)**: Se utiliza para reemplazar el elemento en la posición especificada por el índice con el elemento especificado tomando como argumentos el índice donde se va a realizar la sustitución y el nuevo elemento, y devolviendo el elemento anterior en la posición especificada.
- **indexOf(Object objeto)**: Se usa para obtener el índice de la primera ocurrencia del objeto especificado en la lista tomando como argumento el objeto cuyo índice se desea obtener y devolviendo el índice de la primera ocurrencia del objeto, o -1 si el objeto no está presente en la lista.
- **subList(intfromIndex, inttoIndex)**: Se emplea para obtener una vista de sublista de la lista original, desde el índice fromIndex (inclusive) hasta el índice toIndex (exclusive) tomando como argumentos los índices de inicio y fin de la sublista y devolviendo una lista que es una vista de la sublista de la lista original.

La siguiente imagen (figura 7) muestra el código empleado para hacer uso de los distintos métodos de la interfaz List:

```java
//add
List<String> lista = new ArrayList<>();
lista.add("Hola");
lista.add("Mundo");

//remove
List<String> lista = new ArrayList<>();
lista.add("Hola");
lista.add("Mundo");
lista.remove("Hola");

//get
List<String> lista = new ArrayList<>();
lista.add("Hola");
lista.add("Mundo");
String elemento = lista.get(0); // elemento será "Hola"

//set
List<String> lista = new ArrayList<>();
lista.add("Hola");
lista.add("Mundo");
String elementoAnterior = lista.set(0, "Saludos"); // elementoAnterior será "Hola"

//indexOf
List<String> lista = new ArrayList<>();
lista.add("Hola");
lista.add("Mundo");
int indice = lista.indexOf("Mundo"); // indice será 1

//subList
List<String> lista = new ArrayList<>();
lista.add("H");
lista.add("o");
lista.add("l");
lista.add("a");
List<String> subLista = lista.subList(1, 3); // subLista contendrá ["o", "l"]
```

Fig.7. Ejemplos de uso de los métodos de la interfaz List.

> **¡! Sabías que...**
>
> Las implementaciones de la interfaz List, como ArrayList y LinkedList, proporcionan diferentes características de rendimiento y uso de memoria, lo que las hace adecuadas para diferentes situaciones y requisitos de aplicación.

6.1. ArrayList

Es una **implementación de la interfaz List que utiliza un array dinámico para almacenar los elementos**. Veamos algunas características clave de ArrayList:

- **Acceso rápido**: Los elementos se pueden acceder rápidamente mediante sus índices.
- **Rendimiento de acceso aleatorio**: Ofrece un buen rendimiento para el acceso aleatorio y la recuperación de elementos por índice.
- **Modificaciones costosas**: Las inserciones y eliminaciones en el medio de un ArrayList pueden ser costosas, ya que pueden requerir desplazar los elementos posteriores para hacer espacio para el nuevo elemento o llenar el espacio dejado por el elemento eliminado.

El siguiente código (figura 8) muestra un ejemplo de uso:

```java
import java.util.ArrayList;
import java.util.List;

public class EjemploArrayList {
    public static void main(String[] args) {
        // Crear una instancia de ArrayList para almacenar números enteros
        List<Integer> numeros = new ArrayList<>();

        // Agregar elementos al ArrayList
        numeros.add(10);
        numeros.add(20);
        numeros.add(30);

        // Mostrar el contenido del ArrayList
        System.out.println("Contenido del ArrayList: " + numeros); // Salida: [10, 20, 30]

        // Obtener el tamaño del ArrayList
        int tamaño = numeros.size();
        System.out.println("Tamaño del ArrayList: " + tamaño); // Salida: 3

        // Acceder a elementos específicos del ArrayList
        int primerElemento = numeros.get(0);
        System.out.println("Primer elemento del ArrayList: " + primerElemento); // Salida: 10

        // Modificar un elemento del ArrayList
        numeros.set(1, 25);
        System.out.println("ArrayList después de modificar: " + numeros); // Salida: [10, 25, 30]

        // Eliminar un elemento del ArrayList
        numeros.remove(2);
        System.out.println("ArrayList después de eliminar: " + numeros); // Salida: [10, 25]

        // Verificar si el ArrayList contiene un elemento específico
        boolean contieneElemento = numeros.contains(20);
        System.out.println("¿Contiene el ArrayList el elemento 20?: " + contieneElemento); // Salida: false

        // Recorrer el ArrayList e imprimir cada elemento
        System.out.println("Elementos del ArrayList:");
        for (Integer num : numeros) {
            System.out.println(num);
        }
    }
}
```

Fig.8. Ejemplo de uso de ArrayList.

6.2. LinkedList

Es otra **implementación de la interfaz List que utiliza una estructura de lista doblemente enlazada para almacenar los elementos**. Algunas características clave de LinkedList son:

- **Inserciones y eliminaciones eficientes**: Ofrece un mejor rendimiento para operaciones de inserción y eliminación en comparación con ArrayList.
- **Acceso secuencial menos eficiente**: El acceso secuencial a los elementos en una LinkedList puede ser menos eficiente que en un ArrayList.
- **Uso de memoria**: Utiliza más memoria que ArrayList debido a la sobrecarga de almacenar referencias adicionales para los enlaces entre los nodos.

El siguiente código (figura 9) muestra un ejemplo de uso:

```java
import java.util.LinkedList;

public class EjemploLinkedList {
    public static void main(String[] args) {
        // Crear una instancia de LinkedList para almacenar nombres de personas
        LinkedList<String> nombres = new LinkedList<>();

        // Agregar elementos al final de la lista
        nombres.add("Juan");
        nombres.add("María");
        nombres.add("Pedro");

        // Mostrar el contenido de la lista
        System.out.println("Contenido de la lista: " + nombres); // Salida: [Juan, María, Pedro]

        // Agregar un elemento al principio de la lista
        nombres.addFirst("Luis");
        System.out.println("Lista después de agregar al principio: " + nombres); // Salida: [Luis, Juan, María, Pedro]

        // Agregar un elemento en una posición específica
        nombres.add(2, "Ana");
        System.out.println("Lista después de agregar en la posición 2: " + nombres); // Salida: [Luis, Juan, Ana, María, Pedro]

        // Obtener y eliminar el primer elemento de la lista
        String primerNombre = nombres.removeFirst();
        System.out.println("Primer elemento removido de la lista: " + primerNombre); // Salida: Luis
        System.out.println("Lista después de eliminar el primer elemento: " + nombres); // Salida: [Juan, Ana, María, Pedro]

        // Obtener y eliminar el último elemento de la lista
        String ultimoNombre = nombres.removeLast();
        System.out.println("Último elemento removido de la lista: " + ultimoNombre); // Salida: Pedro
        System.out.println("Lista después de eliminar el último elemento: " + nombres); // Salida: [Juan, Ana, María]

        // Verificar si la lista contiene un elemento específico
        boolean contieneAna = nombres.contains("Ana");
        System.out.println("¿La lista contiene el nombre Ana?: " + contieneAna); // Salida: true

        // Obtener el tamaño de la lista
        int tamaño = nombres.size();
        System.out.println("Tamaño de la lista: " + tamaño); // Salida: 3
    }
}
```

Fig.9. Ejemplo de uso de ArrayList.

Audio 1. "Diferencias entre ArrayList y LinkedList"

https://on.soundcloud.com/zQvwTvq4zgwSd9fg8

/ 7. Conjuntos: Interfaz Set

La interfaz Set en Java representa **una colección que no permite elementos duplicados**.

Las características clave de la interfaz Set son:

- **Elementos únicos**: Los elementos en un conjunto son únicos, lo que significa que no puede haber duplicados en un conjunto.
- **No hay orden garantizado**: Los elementos en un conjunto no tienen un orden específico. La implementación concreta puede almacenar los elementos en cualquier orden y este orden puede cambiar con el tiempo.
- **Operaciones de conjuntos**: La interfaz Set proporciona operaciones comunes de conjuntos, como agregar elementos, eliminar elementos, verificar la existencia de elementos y realizar operaciones de conjuntos como unión, intersección y diferencia.
- **Implementaciones**: Java proporciona varias implementaciones de la interfaz Set, como HashSet, TreeSet y LinkedHashSet, que difieren en sus características y rendimiento.

7.1. HashSet

Se trata de una implementación de la **interfaz Set que utiliza una tabla hash para almacenar elementos**. Algunas características clave de HashSet incluyen:

- **Rendimiento de búsqueda rápido**: Debido a su estructura de tabla *hash*, ofrece un excelente rendimiento para operaciones de búsqueda (*contains*, *add*, *remove*), con complejidad O(1) en promedio.
- **No hay orden garantizado**: Los elementos no tienen un orden específico. La iteración sobre un HashSet puede producir resultados en un orden diferente al orden en que se insertaron los elementos.
- **Elementos únicos**: Los elementos son únicos. Si intentamos agregar un elemento que ya está presente en el conjunto, la operación de agregar no tendrá efecto.
- **Implementación eficiente**: HashSet es una de las implementaciones de conjuntos más eficientes en Java y se utiliza comúnmente cuando se necesitan elementos únicos sin un orden específico.

La siguiente imagen (figura 10) muestra un ejemplo de uso:

```java
import java.util.HashMap;
import java.util.Map;

public class EjemploHashMap {
    public static void main(String[] args) {
        // Crear una instancia de HashMap para almacenar nombres y edades
        Map<String, Integer> mapa = new HashMap<>();

        // Agregar elementos al HashMap
        mapa.put("Jose Antonio", 30);
        mapa.put("María",30);
        mapa.put("Fran", 33);

        // Mostrar el contenido del HashMap
        System.out.println("Contenido del HashMap: " + mapa); // Salida: {Jose Antonio=30, Fran=33, María=30}

        // Obtener y mostrar la edad de Pedro
        int edadDePedro = mapa.get("Fran");
        System.out.println("Edad de Fran: " + edadDeFran); // Salida: 33

        // Verificar si el HashMap contiene a María
        boolean contieneMaria = mapa.containsKey("María");
        System.out.println("¿El HashMap contiene a María?: " + contieneMaria); // Salida: true

        // Eliminar a Juan del HashMap
        mapa.remove("Jose Antonio");
        System.out.println("HashMap después de eliminar a Jose Antonio: " + mapa); // Salida: {Fran=33, María=30}
    }
}
```

Fig.10. Ejemplo de uso de HashSet.

7.2. TreeSet

Se trata de una implementación de la **interfaz Set que emplea un árbol binario balanceado para almacenar los elementos**. Algunas características clave de TreeSet son:

- **Orden natural**: Los elementos están ordenados según su orden natural. Si estos implementan la interfaz Comparable, se utiliza su orden natural para la clasificación.
- **Rendimiento de búsqueda eficiente**: Debido a su estructura de árbol, TreeSet ofrece un buen rendimiento para operaciones de búsqueda, inserción y eliminación.
- **Elementos únicos**: Al igual que otras implementaciones de conjuntos, los elementos en un TreeSet son únicos.
- **Implementación de conjunto ordenado**: TreeSet es una opción ideal cuando se necesita un conjunto ordenado de elementos según un criterio específico, como orden alfabético o numérico.

La siguiente imagen (figura 11) muestra un ejemplo de uso:

```java
import java.util.TreeSet;
import java.util.Set;

public class EjemploTreeSet {
    public static void main(String[] args) {
        // Crear una instancia de TreeSet para almacenar números enteros
        Set<Integer> numeros = new TreeSet<>();

        // Agregar elementos al TreeSet
        numeros.add(10);
        numeros.add(5);
        numeros.add(15);
        numeros.add(3);

        // Mostrar el contenido del TreeSet
        System.out.println("Contenido del TreeSet: " + numeros); // Salida: [3, 5, 10, 15]

        // Verificar si el TreeSet contiene un elemento específico
        boolean contieneCinco = numeros.contains(5);
        System.out.println("¿El TreeSet contiene el número 5?: " + contieneCinco); // Salida: true

        // Eliminar un elemento del TreeSet
        numeros.remove(10);
        System.out.println("TreeSet después de eliminar el número 10: " + numeros); // Salida: [3, 5, 15]
    }
}
```

Fig.11. Ejemplo de uso de TreeSet.

7.3. LinkedHashSet

Se trata de una **implementación de la interfaz Set que combina las características de HashSet y LinkedList**. Algunas características clave de LinkedHashSet son:

- **Orden de inserción**: A diferencia de lo que ocurre en HashSet, los elementos se mantienen en el orden en que fueron insertados.
- **Rendimiento similar a HashSet**: En términos de rendimiento, LinkedHashSet es muy similar a HashSet para operaciones de búsqueda, inserción y eliminación.
- **Elementos únicos**: Al igual que otras implementaciones de conjuntos, los elementos, aquí, son únicos.
- **Implementación de conjunto con orden de inserción**: LinkedHashSet es útil cuando se necesita un conjunto que mantenga el orden de inserción de los elementos, aprovechando la eficiencia de las operaciones de HashSet.

La siguiente imagen (figura 12) muestra un ejemplo de uso:

```java
import java.util.LinkedHashSet;
import java.util.Set;

public class EjemploLinkedHashSet {
    public static void main(String[] args) {
        // Crear una instancia de LinkedHashSet para almacenar nombres de ciudades
        Set<String> ciudades = new LinkedHashSet<>();

        // Agregar elementos al LinkedHashSet
        ciudades.add("Londres");
        ciudades.add("París");
        ciudades.add("Nueva York");
        ciudades.add("Tokio");

        // Mostrar el contenido del LinkedHashSet
        System.out.println("Contenido del LinkedHashSet: " + ciudades); // Salida: [Londres, París, Nueva York, Tokio]

        // Verificar si el LinkedHashSet contiene un elemento específico
        boolean contieneParis = ciudades.contains("París");
        System.out.println("¿El LinkedHashSet contiene París?: " + contieneParis); // Salida: true

        // Eliminar un elemento del LinkedHashSet
        ciudades.remove("Nueva York");
        System.out.println("LinkedHashSet después de eliminar Nueva York: " + ciudades); // Salida: [Londres, París, Tokio]
    }
}
```

Fig.12. Ejemplo de uso de LinkedHashSet.

/ 8. Interfaz Map

La interfaz Map en Java **representa una colección de pares clave-valor, donde cada clave está asociada a un único valor**. Algunas características fundamentales de la interfaz Map son:

- **Pares clave-valor**: Los elementos se almacenan como pares clave-valor, donde cada clave está asociada a un único valor. Las claves deben ser únicas en un Map, pero los valores pueden ser duplicados.
- **Acceso eficiente**: Se puede acceder a los elementos eficientemente mediante sus claves permitiendo recuperar el valor asociado con una clave específica.
- **No hay orden específico**: A diferencia de las colecciones ordenadas como List o LinkedHashSet, los Map no mantienen un orden específico de sus elementos. Sin embargo, algunas implementaciones de Map, como LinkedHashMap, pueden mantener el orden de inserción.
- **Implementaciones**: Java proporciona varias implementaciones de la interfaz Map, como HashMap, TreeMap, LinkedHashMap, etc., que difieren en sus características y comportamientos.

Algunos de los métodos comunes proporcionados por la interfaz Map incluyen put(K clave, V valor), get(Object clave), remove(Object clave), containsKey(Object clave), containsValue(Object valor), keySet(), values(), entrySet(), entre otros.

La siguiente imagen (figura 13) muestra un ejemplo de uso:

```java
import java.util.HashMap;
import java.util.Map;

public class EjemploHashMap {
    public static void main(String[] args) {
        // Crear una instancia de HashMap para almacenar nombres y edades
        Map<String, Integer> mapa = new HashMap<>();

        // Agregar elementos al HashMap
        mapa.put("Jose Antonio", 30);
        mapa.put("María",30);
        mapa.put("Fran", 33);

        // Mostrar el contenido del HashMap
        System.out.println("Contenido del HashMap: " + mapa); // Salida: {Jose Antonio=30, Fran=33, María=30}

        // Obtener y mostrar la edad de Pedro
        int edadDePedro = mapa.get("Fran");
        System.out.println("Edad de Fran: " + edadDeFran); // Salida: 33

        // Verificar si el HashMap contiene a María
        boolean contieneMaria = mapa.containsKey("María");
        System.out.println("¿El HashMap contiene a María?: " + contieneMaria); // Salida: true

        // Eliminar a Juan del HashMap
        mapa.remove("Jose Antonio");
        System.out.println("HashMap después de eliminar a Jose Antonio: " + mapa); // Salida: {Fran=33, María=30}
    }
}
```

Fig.13. Ejemplo de uso de HashMap.

En este ejemplo, se crea un HashMap llamado mapa para almacenar nombres como claves y edades como valores. Luego se agregan elementos al mapa con el método put(), se accede a un valor específico con el método get(), se verifica la existencia de una clave con el método containsKey() y se elimina un elemento con el método remove().

El uso de Map es fundamental en muchos programas Java, ya que proporciona una forma eficiente de asociar datos relacionados entre sí mediante claves y valores. Las implementaciones de Map son ampliamente utilizadas en una variedad de aplicaciones, como la gestión de configuraciones, el almacenamiento de datos en caché, la indexación de información, etc.

En el siguiente recurso podrás aprender más sobre el apartado anterior.

¡! Sabías que...

Existen diferentes tipos de mapas en Java, cada uno con sus propias características y usos específicos. Algunos de los más comunes son: HashMap, TreeMap, LinkedHashMap, EnumMap y ConcurrentHashMap.

/ 9. Caso práctico 2: "Eliminar elementos de la colección"

Planteamiento: Las colecciones nos permiten almacenar y manipular grupos de objetos de manera eficiente y organizada. Con el uso adecuado de las mismas, podemos mejorar el rendimiento de nuestras aplicaciones y facilitar tareas como la búsqueda, la clasificación y el filtrado de datos. Además, las colecciones ofrecen una amplia gama de métodos y operaciones que simplifican el manejo de datos y nos ahorran tiempo y esfuerzo en la implementación de funcionalidades comunes. Conocer y utilizar correctamente las colecciones nos permite escribir código más limpio, modular y escalable, lo que contribuye a un desarrollo más eficiente y efectivo de nuestras aplicaciones.

Nudo: Implementa una aplicación que inserte 50 números enteros aleatorios entre 1 y 10, que puedan estar repetidos. A continuación, elimina todos los elementos que valen 3 mostrando la colección antes y después de la eliminación.

Desenlace:

```
public class EliminarElementos{

    public static void main(String[] args) {
        Collection<Integer> lista = new ArrayList<>();//admite repetidos
        for (int i = 0; i < 50; i++) {
            lista.add((int) (Math.random() * 10 + 1));
        }
        System.out.println(lista);
        boolean eliminado = lista.remove(3);
        while (eliminado) {
            eliminado = lista.remove(3);
        }
        System.out.println(lista);
    }

}
```

Fig.14. Resolución del caso práctico.

La solución que se ha realizado es la planteada en el código de la figura 14.

En la resolución del ejercicio se podría haber prescindido de la variable 'eliminado' y del propio cuerpo del bucle mediante la sentencia while(lista.remove(3)) o emplear el método global removeAll de la siguiente manera: lista. removeAll(3);

/ 10. Colas: Interfaz Queue

La interfaz Queue en Java **representa una estructura de datos de cola, que sigue el principio FIFO** (*First-In, First-Out*), lo que significa que el elemento que se inserta primero es el primero en ser eliminado. Algunas características clave de la interfaz Queue son:

- **Operaciones de cola**: Define operaciones comunes para trabajar con colas, como offer(E elemento) para agregar un elemento al final de la cola, poll() para eliminar y devolver el elemento en la parte delantera de la cola, y peek() para obtener el elemento en la parte delantera de la cola sin eliminarlo.
- **Iteración**: También hereda los métodos de la interfaz Collection, lo que permite iterar sobre los elementos de la cola utilizando iteradores.
- **No acepta elementos nulos**: Dependiendo de la implementación específica, algunas clases que implementan Queue pueden no permitir elementos nulos.
- **Implementaciones**: Java proporciona varias implementaciones de la interfaz Queue, como LinkedList, PriorityQueue y ArrayDeque, cada una con sus propias características y comportamientos.
- **Colas de prioridad**: La interfaz Queue también se usa comúnmente para implementar colas de prioridad, donde los elementos se eliminan de acuerdo con su prioridad, en lugar de seguir un estricto orden FIFO.

La siguiente imagen (figura 15) muestra un ejemplo de uso:

```java
import java.util.LinkedList;
import java.util.Queue;

public class EjemploQueue {
    public static void main(String[] args) {
        // Crear una instancia de Queue utilizando LinkedList
        Queue<String> cola = new LinkedList<>();

        // Agregar elementos a la cola
        cola.offer("Elemento 1");
        cola.offer("Elemento 2");
        cola.offer("Elemento 3");

        // Mostrar el contenido de la cola
        System.out.println("Contenido de la cola: " + cola); // Salida: [Elemento 1, Elemento 2, Elemento 3]

        // Obtener y eliminar el primer elemento de la cola
        String primerElemento = cola.poll();
        System.out.println("Primer elemento de la cola: " + primerElemento); // Salida: Elemento 1
        System.out.println("Contenido de la cola después de poll(): " + cola); // Salida: [Elemento 2, Elemento 3]

        // Obtener el primer elemento de la cola sin eliminarlo
        String primerElementoSinEliminar = cola.peek();
        System.out.println("Primer elemento de la cola sin eliminar: " + primerElementoSinEliminar); // Salida: Elemento 2
        System.out.println("Contenido de la cola después de peek(): " + cola); // Salida: [Elemento 2, Elemento 3]
    }
}
```

Fig.15. Ejemplo de uso de Queue.

En este ejemplo, se crea una cola utilizando la implementación LinkedList. Se agregan elementos a la cola utilizando el método offer(), se recupera y elimina el primer elemento de la cola empleando el método poll(), y se recupera el primer elemento sin eliminarlo usando el método peek(). La salida del programa muestra cómo cambia el contenido de la cola después de cada operación.

Las colas son útiles en situaciones donde los elementos deben ser procesados en el orden en que llegan, como en la implementación de tareas en un sistema de procesamiento de mensajes, en la gestión de trabajos en un sistema de planificación, o en el procesamiento de eventos en un programa de simulación.

¡! Sabías que...

Existen varias clases que se pueden utilizar para implementar una cola (Queue). Algunas de las comunes son: LinkedList y PriorityQueue.

/ 11. Iteradores

En Java, los iteradores **se utilizan para recorrer y acceder a los elementos de una colección de manera secuencial**. Proporcionan un medio para acceder a los elementos de una colección sin exponer la estructura interna de la misma.

Los iteradores nos van a aportar tres métodos:

- **next()**: Devuelve el valor del elemento en el que está el iterador.
- **hasNext()**: Indica si hay un elemento siguiente en el orden de iteración.
- **remove()**: Eliminará el elemento que tenga el iterador de la colección.

Los iteradores se pueden usar de forma transparente al usuario mediante el bucle 'for-each'. Este tipo de bucle recorre una colección utilizando iteradores, pero sin que el programador tenga que definirlos. Su sintaxis es la siguiente:

```
for( tipo nombrevariable : Colecction )
```

Donde:

- 'tipo' es el tipo de dato que contiene la colección.
- 'nombrevariable' es el nombre de la variable iteradora.
- 'Colecction' es la colección que vamos a recorrer.

La siguiente figura muestra un ejemplo de uso de los iteradores:

```java
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class EjemploIterador {
    public static void main(String[] args) {
        // Crear una lista de cadenas
        List<String> lista = new ArrayList<>();
        lista.add("Uno");
        lista.add("Dos");
        lista.add("Tres");

        // Obtener un iterador para la lista
        Iterator<String> iterador = lista.iterator();

        // Iterar sobre la lista utilizando el iterador
        while (iterador.hasNext()) {
            String elemento = iterador.next();
            System.out.println(elemento);
        }
    }
}
```

Fig.16. Ejemplo de uso de los iteradores.

En este ejemplo:

- Se crea una lista de cadenas llamada 'lista' y se agregan algunos elementos a ella.
- Se obtiene un iterador para la lista utilizando el método 'iterator()' de la interfaz List.

- Se opera sobre la lista utilizando un bucle 'while'. En cada iteración, verificamos si hay más elementos disponibles utilizando el método 'hasNext()' del iterador y, si es así, obtenemos el siguiente elemento utilizando el método 'next()'.

- Se trata de un elemento común en Java para recorrer colecciones como listas, conjuntos y mapas, ya que proporciona un enfoque limpio y seguro para acceder a los elementos de una colección sin preocuparse por los detalles de la implementación subyacente.

Vídeo 2. "Ejemplo de uso de iteradores"
https://bit.ly/2ByjTem

/ 12. Resumen y resolución del caso práctico de la unidad

A lo largo de este tema hemos visto qué son las colecciones, las cuales nos permiten almacenar, dentro de un mismo contenedor, muchos elementos del mismo tipo, y los datos genéricos, que nos permiten crear tanto clases como métodos sin especificar los datos, para más tarde poder usarlos con los tipos de datos que necesitemos.

Hemos profundizado en las colecciones más importantes, que son las listas, los mapas y los conjuntos, cada uno con una serie de propiedades que los hace únicos y capaces de resolver problemas totalmente diferentes.

También hemos aprendido el concepto de iterador y que gracias a él podemos recorrer cualquier tipo de colección, sea cual sea el tipo de dato del que esté formada, de una forma eficiente y segura. Esto es así porque los iteradores implementan métodos para no sobrepasar los límites de las colecciones.

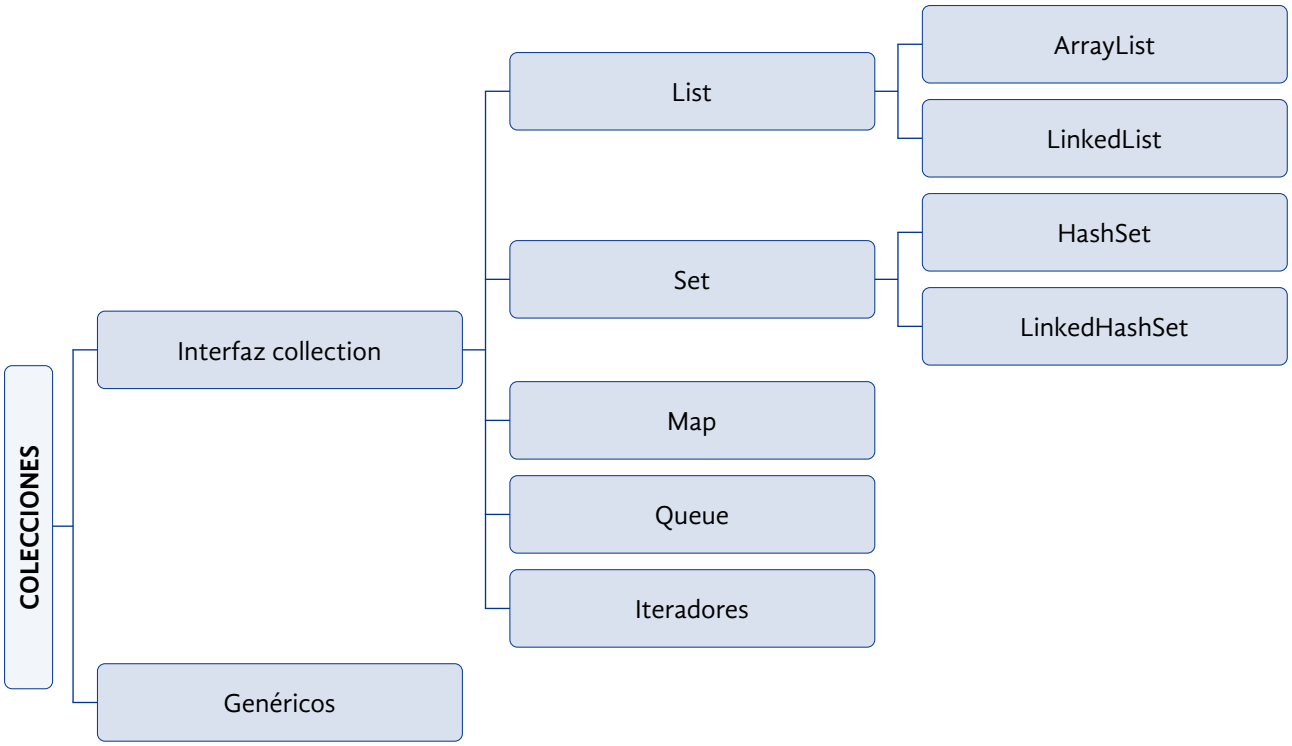


Fig.17. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Una de las ventajas de las colecciones es su gran número de elementos. Estos nos pueden sacar de un aprieto si sabemos cómo utilizarlos, pero claro, esto puede ser un arma de doble filo, ya que, al haber tantas opciones para elegir y cada una de ellas tener tantos métodos diferentes, puede que no sepamos por cuál decidirnos, terminando así utilizando la más básica, los arrays convencionales.

Una muy buena práctica sería anotar, en una lista, los distintos tipos de colecciones disponibles junto a su utilidad, para poder consultar dicha lista y decidir de una forma más fácil, ya que podríamos comparar cada nuevo caso que se nos presente con los ejemplos de ese listado y así establecer un símil que nos ayude a decidirnos.

Un ejemplo de esto podría ser que necesitáramos obtener una cantidad de números no repetidos, para lo cual podríamos utilizar un HashSet, ya que por definición este no deja tener números duplicados; de este modo, evitaríamos utilizar un array y tener que gestionar la no repetición de elementos.

Las listas, las podríamos emplear para almacenar datos que acepten números repetidos, como, por ejemplo, las notas de una clase. Los mapas los podremos usar cuando necesitemos acceder rápidamente a un dato ligado a una clave, como, por ejemplo, los datos de una persona, siendo el DNI su clave.

/ 13. Bibliografía

Amor, R. V. (2020, 6 de marzo). Introducción a Colecciones en Java - Adictos al trabajo. *Adictos Al Trabajo*. Recuperado de: https://adictosaltrabajo.com/2015/09/25/introduccion-a-colecciones-en-java/

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Flujos de datos (s. f.). Recuperado de: http://www.sc.ehu.es/sbweb/fisica/cursoJava/fundamentos/archivos/flujos.htm

Knuth, D. E. (1997). *The Art of Computer Programming (Vol. 1: Fundamental Algorithms)* (3.ª ed.). Addison-Wesley.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S.A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference* (11.ª ed.). McGraw-Hill Education.